

文本匹配小案例四：多维评分与可解释匹配

一、本节课要解决什么问题

在“小案例二”和“小案例三”中，我们已经学会了使用：

- 词袋模型；
- TF-IDF；
- Word2Vec；
- 余弦相似度；
- 简历与岗位相似度排序。

这些方法可以帮助我们回答一个问题：

这份简历和这个岗位在文本内容上像不像？

但是，真实的人岗匹配不只看“文本像不像”。

比如有两个岗位：

岗位 A：Python 数据分析实习生，要求本科，经验不限，地点南昌

岗位 B：Python 高级数据工程师，要求硕士，3 年经验，地点北京

一名学生简历中写了：

熟悉 Python、SQL、数据分析，做过课程项目，本科，应届生，希望在南昌工作

从文本关键词看，学生和两个岗位都可能有不少共同词。但从实际匹配看，岗位 A 更合适，因为它的学历、经验、城市要求更接近学生情况。

所以本节课要解决的问题是：

在文本相似度之外，加入技能、学历、经验、城市、薪资、证书等维度，得到更合理、更能解释的综合匹配分。

本节课的目标不是做一个“神秘黑盒模型”，而是做一个学生能看懂、教师能讲清、项目能展示的可解释匹配系统。

二、本节课的核心思想

1. 为什么不能只看文本相似度

只看 TF-IDF、Word2Vec 或余弦相似度这类文本相似度，仍然可能会遇到这些问题：

- 简历和岗位都提到 `Python`，但岗位要求 3 年经验，学生没有经验；
- 简历和岗位都提到 `数据分析`，但岗位要求硕士，学生是本科；
- 简历和岗位关键词很像，但城市和薪资期望差距很大；
- 简历很长，关键词很多，看起来什么都沾一点，但关键技能不一定匹配；
- 岗位描述很短，文本相似度可能偏低，但核心技能刚好匹配。

这就像点外卖不能只看菜名。

你看到两个套餐：

套餐 A：鸡腿饭，18 元，距离 800 米，预计 25 分钟送达

套餐 B：鸡腿饭，38 元，距离 5 公里，预计 60 分钟送达

如果只看菜名，它们都叫鸡腿饭，好像差不多。

但真实选择时，你还会考虑：

- 价格；
- 距离；
- 配菜；
- 评价；
- 送达时间；
- 自己现在饿不饿。

简历匹配也是一样。

文本相似度只是一个维度，真实推荐需要多维判断。

2. 什么是多维评分

多维评分就是把多个判断角度分别打分，再按照权重合成总分。

例如：

维度	说明	示例权重
技能匹配分	简历技能和岗位技能重合程度	40%
语义相似度分	综合 TF-IDF 和 Word2Vec 后得到的文本方向相似程度	30%
学历匹配分	学生学历是否满足岗位要求	15%
经验匹配分	学生经验年限是否满足岗位要求	15%

综合分可以写成：

$$\text{综合匹配分} = \text{技能分} \times 0.40 + \text{语义分} \times 0.30 + \text{学历分} \times 0.15 + \text{经验分} \times 0.15$$

这里的“语义分”不是凭空来的，它可以由前两节课的结果合成：

$$\text{语义分} = \text{TF-IDF 相似度分} \times 0.60 + \text{Word2Vec 相似度分} \times 0.40$$

也就是说，TF-IDF 更关注词面重合，Word2Vec 更关注上下文语义，两者先合成一个统一的 `semantic_score`，再进入多维评分。

这就是教学计划中提到的核心公式：

```
# semantic_score 可以由 TF-IDF 分和 Word2Vec 分加权合成
semantic_score = tfidf_score * 0.60 + word2vec_score * 0.40

# 综合匹配分再把技能、语义、学历、经验四个维度合起来
total_score = skill_score * 0.40 + semantic_score * 0.30 + education_score * 0.15 + experience_score * 0.15
```

3. 什么是可解释匹配

可解释匹配不是只输出：

匹配分：82.5

而是要告诉用户：

推荐原因：

1. 技能匹配较高，共同技能包括 Python、SQL、数据分析；
2. 学历满足岗位要求；
3. 经验略低于岗位要求；
4. 文本相似度较高，说明简历内容和岗位描述方向接近。

一个好的简历匹配系统，最好能同时回答三个问题：

1. 匹配多少分？
2. 为什么是这个分？
3. 哪些地方匹配，哪些地方还不够？

这就是本节课的重点。

三、课前准备

1. 需要的 Python 包

本节课主要使用 Python 基础语法、`pandas`、`scikit-learn` 和 `gensim`。

如果要运行最后的综合案例，还会用到 `gensim` 中的 `Word2Vec`。

```
pip install pandas scikit-learn gensim numpy -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
```

2. 建议创建的代码文件

建议创建文件夹：

```
text_matching_demo_04
```

本讲义中的代码可以保存为：

```
01_skill_score_demo.py  
02_education_score_demo.py  
03_experience_score_demo.py  
04_extra_rule_score_demo.py  
05_weighted_score_demo.py  
06_explain_reason_demo.py  
07_batch_score_dataframe.py  
08_weight_adjust_compare.py  
09_integrated_explainable_matcher.py
```

四、先认识一份简历和一个岗位

为了让问题简单清楚，我们先用字典表示简历和岗位。

示例数据

resume 表示一份学生简历。
在真实项目中，这些字段通常来自 Excel、CSV、数据库，或者简历解析结果。

```
resume = {  
    # 学生姓名，用于展示结果  
    "name": "张同学",  
  
    # 学生已经掌握的技能，通常来自分词、技能词标准化或人工整理  
    "skills": ["python", "sql", "数据分析", "可视化"],  
  
    # 学生最高学历，后面会被转换成数字等级进行比较  
    "education": "本科",  
  
    # 学生相关经验年限。应届生或没有正式工作经验可以写 0  
    "experience_years": 0,  
  
    # 学生期望工作城市  
    "city": "南昌",  
  
    # 学生期望薪资，单位可以约定为元/月  
    "expected_salary": 5000,  
  
    # 学生拥有的证书  
    "certificates": ["英语四级"],  
  
    # 简历中的文本描述，用于 TF-IDF、Word2Vec 等文本相似度计算  
    "text": "熟悉 Python、SQL 和数据分析，做过可视化课程项目"  
}
```

job 表示一个招聘岗位。
它和 resume 使用相近的字段，方便后面一项一项比较。

```
job = {  
    # 岗位名称，用于展示推荐结果
```

```
"job_name": "数据分析实习生",

# 岗位要求的核心技能
"required_skills": ["python", "sql", "数据分析"],

# 岗位最低学历要求
"min_education": "本科",

# 岗位最低经验要求
"min_experience_years": 0,

# 岗位所在城市
"city": "南昌",

# 岗位薪资, 单位与 expected_salary 保持一致
"salary": 6000,

# 岗位偏好的证书, 不一定是硬性要求
"preferred_certificates": ["英语四级"],

# 岗位描述文本, 用于和简历文本计算相似度
"text": "要求熟悉 Python、SQL, 能够完成数据分析和报表制作"
}
```

这份数据可以理解为：

字段	含义
skills	学生简历中提取出来的技能
required_skills	岗位要求的技能
education	学生学历
min_education	岗位最低学历要求
experience_years	学生经验年限
min_experience_years	岗位最低经验要求
city	城市
expected_salary	学生期望薪资
salary	岗位薪资
certificates	学生证书
preferred_certificates	岗位偏好的证书
text	用于文本相似度计算的文本

本节课后面的案例，就是围绕这些字段逐步展开。

思考题 1

为什么要把简历和岗位整理成字典，而不是一直使用一大段文字？

参考答案：

因为一大段文字适合做文本相似度，但不方便分别判断学历、经验、城市、薪资等结构化信息。字典可以把不同字段拆开，每个字段用适合自己的方法评分。

五、技能匹配分：先看“会什么”和“要什么”

1. 技能匹配为什么重要

在简历匹配中，技能通常是最核心的维度。

岗位要求：

python、sql、数据分析

学生掌握：

python、sql、可视化

共同技能是：

python、sql

这说明学生已经满足岗位的一部分要求。

生活类比：

如果你要做一道番茄炒蛋，菜谱要求：

番茄、鸡蛋、盐、油

厨房里有：

番茄、鸡蛋、酱油、米饭

你至少已经具备主要材料，虽然还缺盐和油。技能匹配也是类似的“材料检查”。

2. 技能匹配分的简单公式

一种简单算法是：

技能匹配分 = 共同技能数量 / 岗位要求技能数量

如果岗位要求 4 个技能，学生掌握其中 3 个：

技能匹配分 = $3 / 4 = 0.75$

为了方便展示，可以乘以 100：

技能匹配分 = 75 分

六、案例 1：计算技能匹配分

代码清单：01_skill_score_demo.py

```
# 01_skill_score_demo.py
# 目标：计算简历技能与岗位技能的匹配程度

# 学生简历中提取出的技能列表
resume_skills = ["python", "sql", "数据分析", "可视化"]

# 岗位要求的技能列表
job_required_skills = ["python", "sql", "数据分析", "excel"]

def calculate_skill_score(resume_skills, job_required_skills):
    """
    计算技能匹配分。

    思路：
    1. 把两个技能列表转换成集合，方便求交集。
    2. 共同技能数量 / 岗位要求技能数量。
    3. 返回 0 到 100 之间的分数。
    """
    resume_skill_set = set(resume_skills)
    job_skill_set = set(job_required_skills)

    # 交集表示双方都包含的技能
    matched_skills = resume_skill_set & job_skill_set

    # 差集表示岗位要求但简历没有的技能
    missing_skills = job_skill_set - resume_skill_set

    # 如果岗位没有填写技能要求，不能直接除以 0
    # 这里简单处理为 0 分，也可以根据业务规则改为默认分
    if len(job_skill_set) == 0:
        score = 0
```

```
    else:
        score = len(matched_skills) / len(job_skill_set) * 100

    return score, matched_skills, missing_skills

score, matched, missing = calculate_skill_score(resume_skills, job_required_skills)

print("技能匹配分: ", round(score, 2))
print("共同技能: ", matched)
print("缺少技能: ", missing)
```

运行结果解读

学生技能:

python、sql、数据分析、可视化

岗位要求:

python、sql、数据分析、excel

共同技能:

python、sql、数据分析

岗位要求 4 个技能，学生满足 3 个，所以：

$$\text{技能匹配分} = 3 / 4 \times 100 = 75$$

思考题 2

如果学生掌握很多额外技能，但岗位要求的核心技能一个都不会，技能匹配分应该高吗？

参考答案：

不应该高。因为技能匹配分主要看岗位要求是否被满足。额外技能可以作为加分项，但不能替代岗位核心技能。

七、学历匹配分：是否达到门槛

1. 学历评分的基本思想

学历不是越高就一定越适合，但很多岗位会设置最低学历要求。

例如：

岗位要求：本科及以上
学生学历：大专

这种情况通常不完全匹配。

如果：

岗位要求：本科及以上
学生学历：本科

则满足要求。

如果：

岗位要求：本科及以上
学生学历：硕士

也满足要求。

2. 把学历转换成等级

为了方便比较，可以把学历映射成数字：

学历	等级
高中	1
大专	2
本科	3
硕士	4
博士	5

如果学生学历等级大于等于岗位要求等级，就给满分。

八、案例 2：计算学历匹配分

代码清单: 02_education_score_demo.py


```
# 02_education_score_demo.py
# 目标：判断学生学历是否满足岗位最低学历要求

# 把文字学历映射成数字等级。
# 数字越大，表示学历层次越高。
# 这样 Python 才能比较“本科”和“硕士”谁的等级更高。
education_level = {
    "高中": 1,
    "大专": 2,
    "本科": 3,
    "硕士": 4,
    "博士": 5,
}

def calculate_education_score(resume_education, job_min_education):
    """
    计算学历匹配分。

    规则：
    1. 学生学历 >= 岗位最低学历要求：100 分。
    2. 学生学历比岗位要求低 1 级：60 分。
    3. 学生学历比岗位要求低 2 级或更多：30 分。
    """
    # get(key, 0) 表示：
    # 如果字典中能找到这个学历，就返回对应等级；
    # 如果找不到，就返回 0，避免程序报错。
    resume_level = education_level.get(resume_education, 0)
    job_level = education_level.get(job_min_education, 0)

    # 学生学历等级大于等于岗位要求等级，说明满足门槛
    if resume_level >= job_level:
```

```
        return 100

    # gap 表示学生学历距离岗位要求还差几个等级
    gap = job_level - resume_level

    # 只低一级，说明差距不大，给 60 分
    if gap == 1:
        return 60

    # 低两级或更多，说明差距较明显，给 30 分
    return 30

# 下面用几组测试数据观察评分结果
print("本科 vs 本科：", calculate_education_score("本科", "本科"))
print("大专 vs 本科：", calculate_education_score("大专", "本科"))
print("本科 vs 硕士：", calculate_education_score("本科", "硕士"))
print("硕士 vs 本科：", calculate_education_score("硕士", "本科"))
```

运行结果解读

- 本科满足本科要求，得 100；
- 大专比本科低一级，得 60；
- 本科比硕士低一级，得 60；
- 硕士高于本科要求，得 100。

这里的 60 和 30 不是固定真理，而是我们设计的业务规则。

真实项目中，评分规则可以根据课程要求、企业要求或数据情况调整。

讨论题 1

如果一个岗位写“学历不限”，学历匹配分应该怎么处理？

参考答案：

可以直接给 100 分，因为岗位没有设置学历门槛。也可以在数据预处理时把“学历不限”转换成最低等级，例如“高中”或“不限”，再单独写规则处理。

九、经验匹配分：是否接近岗位要求

1. 经验评分的基本思想

经验年限通常是一个数字。

例如：

学生经验：0 年
岗位要求：1 年

这种情况并不一定完全不匹配。对于实习岗位，学生没有正式工作经验也可能可以接受。

所以经验评分可以设计得柔和一点：

情况	分数
学生经验 $\geq$ 岗位要求	100
学生经验比岗位要求少 1 年	70
学生经验比岗位要求少 2 年	40
差距更大	20

生活类比：

学骑自行车时，已经练过 3 次的人，不等于完全熟练，但比一次没练过更接近要求。

经验评分也是看“距离要求还有多远”。

十、案例 3：计算经验匹配分

代码清单： `03_experience_score_demo.py`

```
# 03_experience_score_demo.py
```

```
# 目标：根据经验年限计算匹配分
```

```
def calculate_experience_score(resume_years, job_min_years):
```

```
    """
```

```
    计算经验匹配分。
```

```
    参数：
```

```
        resume_years: 简历中的经验年限
```

```
        job_min_years: 岗位要求的最低经验年限
```

```
    返回：
```

```
        0 到 100 之间的分数
```

```
    """
```

```
    # 学生经验年限达到或超过岗位要求，直接给满分
```

```
    if resume_years >= job_min_years:
```

```
        return 100
```

```
    # gap 表示学生距离岗位要求还差多少年经验
```

```
    gap = job_min_years - resume_years
```

```
    # 只差 1 年，说明比较接近岗位要求
```

```
    if gap == 1:
```

```
        return 70
```

```
    # 差 2 年，说明有一定距离，但不是完全不相关
```

```
    elif gap == 2:
```

```
        return 40
```

```
    # 差距更大时给较低分
```

```
    else:
```

```
        return 20
```

```
# 测试用例，每个元组表示：
# (学生经验年限，岗位要求经验年限)
test_cases = [
    (0, 0),
    (0, 1),
    (1, 3),
    (1, 5),
    (3, 2),
]

# 逐组测试经验评分函数
for resume_years, job_years in test_cases:
    score = calculate_experience_score(resume_years, job_years)
    print(f"学生经验 {resume_years} 年, 岗位要求 {job_years} 年, 经验分: {score}")
```

思考题 3

为什么经验不足时不一定直接给 0 分？

参考答案：

因为经验不足不代表完全不能胜任。尤其是实习岗位、初级岗位，项目经历、学习能力和技能匹配也很重要。给一个较低但不是 0 的分数，可以让综合评分更灵活。

十一、城市、薪资、证书等附加规则

除了技能、学历、经验，还可以加入一些业务规则。

1. 城市匹配

如果学生希望在南昌工作，岗位也在南昌，可以加分。

如果城市不同，但学生接受异地，也可以给部分分。

2. 薪资匹配

如果岗位薪资高于或接近学生期望，说明薪资比较合适。

如果岗位薪资远低于学生期望，匹配度可能下降。

3. 证书匹配

有些岗位偏好：

- 英语四级；
- 计算机二级；
- 普通话证书；
- 数据分析相关证书。

如果学生拥有相关证书，可以作为加分项。

生活类比：

租房时，房子本身很重要，但你也会看：

- 离学校远不远；
- 租金是否能接受；
- 有没有空调和洗衣机；

- 周边是否方便。

这些不是唯一因素，但会影响最后选择。

十二、案例 4：城市、薪资、证书评分

代码清单： `04_extra_rule_score_demo.py`


```
# 04_extra_rule_score_demo.py
```

```
# 目标：计算城市、薪资、证书这些附加维度的评分
```

```
def calculate_city_score(resume_city, job_city, accept_other_city=False):
```

```
    """
```

```
    计算城市匹配分。
```

```
    规则：
```

1. 城市完全相同：100 分。
2. 城市不同，但学生接受异地：70 分。
3. 城市不同，且不接受异地：30 分。

```
    """
```

```
    # 城市完全相同，说明到岗成本较低
```

```
    if resume_city == job_city:
```

```
        return 100
```

```
    # 城市不同，但学生愿意接受异地，可以给部分分
```

```
    if accept_other_city:
```

```
        return 70
```

```
    # 城市不同且不接受异地，匹配度较低
```

```
    return 30
```

```
def calculate_salary_score(expected_salary, job_salary):
```

```
    """
```

```
    计算薪资匹配分。
```

```
    规则：
```

1. 岗位薪资 >= 学生期望薪资：100 分。
2. 岗位薪资达到期望的 80% 以上：70 分。

3. 岗位薪资低于期望的 80%: 40 分。

"""

岗位薪资达到或超过学生期望, 直接给满分

```
if job_salary >= expected_salary:  
    return 100
```

如果学生没有填写期望薪资, 无法计算比例, 这里给一个中性分

```
if expected_salary == 0:  
    return 60
```

ratio 表示岗位薪资占学生期望薪资的比例

例如期望 5000, 岗位 4500, 则 ratio = 0.9

```
ratio = job_salary / expected_salary
```

达到期望薪资的 80% 以上, 说明还可以接受

```
if ratio >= 0.8:  
    return 70
```

低于期望薪资较多, 给较低分

```
return 40
```

```
def calculate_certificate_score(resume_certificates, preferred_certificates):
```

"""

计算证书匹配分。

规则:

岗位偏好证书中, 学生拥有的比例越高, 分数越高。

如果岗位没有证书偏好, 默认给 100 分。

"""

转成集合, 方便求共同证书

```
resume_set = set(resume_certificates)
```

```

preferred_set = set(preferred_certificates)

# 岗位没有证书偏好时, 不因为证书扣分
if len(preferred_set) == 0:
    return 100, set()

# matched 表示学生证书与岗位偏好证书的交集
matched = resume_set & preferred_set

# 证书分 = 匹配证书数量 / 岗位偏好证书数量
score = len(matched) / len(preferred_set) * 100

return score, matched

# 分别测试三个附加维度
city_score = calculate_city_score("南昌", "南昌")
salary_score = calculate_salary_score(5000, 4500)
certificate_score, matched_certs = calculate_certificate_score(
    ["英语四级", "计算机二级"],
    ["英语四级"]
)

print("城市分: ", city_score)
print("薪资分: ", salary_score)
print("证书分: ", certificate_score)
print("匹配证书: ", matched_certs)

```

讨论题 2

附加规则是不是越多越好?

参考答案：

不一定。规则太少会粗糙，规则太多会复杂，也可能引入不稳定因素。教学项目中建议先选 4 到 6 个容易解释的维度，例如技能、语义、学历、经验、城市、证书。

十三、综合权重：把多个分数合成一个总分

现在我们有了多个分数：

技能分：75
TF-IDF分：78
Word2Vec分：82
语义分：79.6
学历分：100
经验分：70

如果直接平均：

$$\text{总分} = (75 + 79.6 + 100 + 70) / 4 = 81.15$$

这很简单，但可能不够合理。

因为在简历匹配中，技能往往比城市更重要，学历可能是门槛，经验可能对初级岗位影响较小。

所以我们会引入权重。

1. 什么是权重

权重表示“某个维度在总分中有多重要”。

例如：

技能分权重：40%
语义分权重：30%
学历分权重：15%
经验分权重：15%

说明：

- 技能是最重要的；
- 文本相似度也很重要；
- 学历和经验作为补充判断。

2. 权重需要加起来等于 1

一般来说，所有权重加起来应该等于 1：

$$0.40 + 0.30 + 0.15 + 0.15 = 1.00$$

这样总分仍然保持在 0 到 100 的范围内。

十四、案例 5：计算综合匹配分

代码清单：05_weighted_score_demo.py

```
# 05_weighted_score_demo.py
```

```
# 目标：把多个维度的评分合成为一个综合匹配分
```

```
def calculate_semantic_score(tfidf_score, word2vec_score):
```

```
    """
```

```
    计算语义分。
```

```
    这里把“小案例二”的 TF-IDF 相似度分和“小案例三”的 Word2Vec 相似度分  
    先合成一个统一的 semantic_score。
```

```
    权重设计：
```

- TF-IDF：60%，更关注词面重合。
- Word2Vec：40%，更关注上下文语义接近。

```
    """
```

```
    semantic_score = tfidf_score * 0.60 + word2vec_score * 0.40
```

```
    return semantic_score
```

```
def calculate_total_score(skill_score, semantic_score, education_score, experience_score):
```

```
    """
```

```
    计算综合匹配分。
```

```
    权重设计：
```

- 技能分：40%
- 语义分：30%
- 学历分：15%
- 经验分：15%

```
    """
```

```
    total_score = (  
        skill_score * 0.40
```

```
        + semantic_score * 0.30
        + education_score * 0.15
        + experience_score * 0.15
    )

    return total_score

# 假设前面几个评分函数已经算出了这些分数
skill_score = 75

# TF-IDF 和 Word2Vec 来自前两节文本相似度案例
tfidf_score = 78
word2vec_score = 82

# 学历分和经验分来自本节课前面的规则评分
education_score = 100
experience_score = 70

# 先把两个文本相似度分合成一个语义分
semantic_score = calculate_semantic_score(tfidf_score, word2vec_score)

# 再把语义分和其他结构化维度合成综合分
total = calculate_total_score(
    skill_score,
    semantic_score,
    education_score,
    experience_score
)

print("技能分: ", skill_score)
print("TF-IDF 相似度分: ", tfidf_score)
```

```
print("Word2Vec 相似度分: ", word2vec_score)
print("语义分: ", round(semantic_score, 2))
print("学历分: ", education_score)
print("经验分: ", experience_score)
print("综合匹配分: ", round(total, 2))
```

运行结果解读

计算过程是：

```
78 × 0.60 + 82 × 0.40 = 79.6
75 × 0.40 = 30
79.6 × 0.30 = 23.88
100 × 0.15 = 15
70 × 0.15 = 10.5
总分 = 79.38
```

所以综合匹配分是：

79.38

思考题 4

如果把技能分权重从 40% 改成 50%，会发生什么？

参考答案：

技能分对总分的影响会变大。技能匹配高的岗位更容易排到前面，技能匹配低但学历、经验等其他维度较好的岗位可能会被压下去。

十五、可解释推荐：让系统说清楚为什么推荐

综合分解决了“排第几”的问题，但还没有解决“为什么”的问题。

一个更适合展示和答辩的结果应该像这样：

推荐岗位：数据分析实习生

综合匹配分：84.5

推荐理由：

1. 技能匹配较好，共同技能包括 python、sql、数据分析；
2. 学历满足岗位要求；
3. 经验满足岗位要求；
4. 城市相同，通勤或到岗成本较低；
5. 文本相似度较高，说明简历经历和岗位职责方向接近。

可解释推荐的好处：

- 学生知道自己为什么被推荐；
- 教师能看出算法是否合理；
- 答辩时更容易讲清楚项目；
- 后期可以根据解释发现规则问题。

十六、案例 6：生成匹配理由

代码清单：06_explain_reason_demo.py

```
# 06_explain_reason_demo.py
```

```
# 目标：根据各维度结果，生成用户能看懂的匹配理由
```

```
def generate_match_reason(
    matched_skills,
    missing_skills,
    skill_score,
    semantic_score,
    education_score,
    experience_score,
    city_score
):
    """
    生成匹配理由。

    参数说明：
        matched_skills: 共同技能集合
        missing_skills: 岗位要求但简历缺少的技能集合
        skill_score: 技能匹配分
        semantic_score: 文本相似度分
        education_score: 学历分
        experience_score: 经验分
        city_score: 城市分
    """
    reasons = []

    # 根据技能分生成技能相关解释
    if skill_score >= 80:
        reasons.append(f"技能匹配较高，共同技能包括：{'', '.join(matched_skills)}。")
    elif skill_score >= 50:
        reasons.append(f"技能部分匹配，已匹配技能：{'', '.join(matched_skills)}。")
```

```
else:
    reasons.append("技能匹配偏低，需要补充岗位核心技能。")

# 如果有缺少技能，要单独提示出来，方便学生知道改进方向
if len(missing_skills) > 0:
    reasons.append(f"仍需提升的技能包括: {'', ' '.join(missing_skills)}。")

# 根据语义分解释简历文本和岗位描述是否接近
if semantic_score >= 80:
    reasons.append("文本相似度较高，简历经历与岗位描述方向接近。")
elif semantic_score >= 50:
    reasons.append("文本相似度中等，简历与岗位存在一定相关性。")
else:
    reasons.append("文本相似度较低，简历内容与岗位描述差异较大。")

# 根据学历分解释是否满足学历门槛
if education_score == 100:
    reasons.append("学历满足岗位要求。")
else:
    reasons.append("学历与岗位要求存在差距，需要结合其他优势综合判断。")

# 根据经验分解释工作或项目经验是否接近岗位要求
if experience_score == 100:
    reasons.append("经验年限满足岗位要求。")
elif experience_score >= 70:
    reasons.append("经验略低于岗位要求，但差距不大。")
else:
    reasons.append("经验年限与岗位要求差距较大。")

# 根据城市分解释到岗便利性
if city_score == 100:
    reasons.append("城市匹配，到岗便利性较好。")
```

```

else:
    reasons.append("城市不完全匹配，需要确认是否接受异地。")

# 返回列表形式，后面可以逐条打印，也可以展示到 Streamlit 页面
return reasons

# 假设前面的评分函数已经得到这些结果
matched_skills = {"python", "sql", "数据分析"}
missing_skills = {"excel"}

# 调用解释函数，生成一组推荐理由
reasons = generate_match_reason(
    matched_skills=matched_skills,
    missing_skills=missing_skills,
    skill_score=75,
    semantic_score=82,
    education_score=100,
    experience_score=70,
    city_score=100
)

print("推荐理由：")
# enumerate(reasons, start=1) 可以同时得到序号和理由内容
for index, reason in enumerate(reasons, start=1):
    print(f"{index}. {reason}")

```

运行结果解读

这个案例会输出多条推荐理由，而不是只输出一个分数。

这就是“可解释”的第一步：

把每个维度的判断结果翻译成人能读懂的话。

讨论题 3

为什么可解释推荐对教学项目特别重要？

参考答案：

因为教学项目不仅要跑出结果，还要让学生能讲清楚结果是怎么来的。可解释推荐能帮助学生理解评分规则，也能让教师检查系统是否合理。

十七、案例 7：用 pandas 批量计算多维评分

前面的案例只计算一份简历和一个岗位。

真实项目中通常有多份简历和多个岗位。

我们可以用 `pandas` 把结果整理成表格，方便后续保存 CSV 或展示到 Streamlit。

代码清单： `07_batch_score_dataframe.py`

```
# 07_batch_score_dataframe.py
# 目标：对多份简历和多个岗位批量计算综合匹配分
```

```
import pandas as pd
```

```
# 学历等级表，后面会用数字等级比较学历高低
```

```
education_level = {
    "高中": 1,
    "大专": 2,
    "本科": 3,
    "硕士": 4,
    "博士": 5,
}
```

```
def calculate_skill_score(resume_skills, job_required_skills):
```

```
    """计算技能分，同时返回共同技能和缺少技能。"""
```

```
    # 转成集合后，可以直接用交集和差集处理技能列表
```

```
    resume_set = set(resume_skills)
```

```
    job_set = set(job_required_skills)
```

```
    # 岗位没有填写技能要求时，避免除以 0
```

```
    if len(job_set) == 0:
```

```
        return 0, set(), set()
```

```
    # matched 是共同技能，missing 是岗位要求但学生没有的技能
```

```
    matched = resume_set & job_set
```

```
    missing = job_set - resume_set
```

```
    # 技能分 = 共同技能数量 / 岗位要求技能数量
```

```
    score = len(matched) / len(job_set) * 100
```

```
return score, matched, missing
```

```
def calculate_education_score(resume_education, job_min_education):
```

```
    """计算学历分。"""
```

```
    # 把文字学历转换成数字等级, 未知学历默认为 0
```

```
    resume_level = education_level.get(resume_education, 0)
```

```
    job_level = education_level.get(job_min_education, 0)
```

```
    # 满足最低学历要求, 给满分
```

```
    if resume_level >= job_level:
```

```
        return 100
```

```
    # 只低一级, 给部分分
```

```
    if job_level - resume_level == 1:
```

```
        return 60
```

```
    # 差距更大, 给较低分
```

```
    return 30
```

```
def calculate_experience_score(resume_years, job_min_years):
```

```
    """计算经验分。"""
```

```
    # 达到岗位经验要求, 给满分
```

```
    if resume_years >= job_min_years:
```

```
        return 100
```

```
    # gap 表示差几年
```

```
    gap = job_min_years - resume_years
```

```
    # 按经验差距给不同分数
```

```
if gap == 1:
    return 70
elif gap == 2:
    return 40
else:
    return 20

def calculate_total_score(skill_score, semantic_score, education_score, experience_score):
    """按照固定权重计算综合匹配分。"""
    return (
        skill_score * 0.40
        + semantic_score * 0.30
        + education_score * 0.15
        + experience_score * 0.15
    )

# 为了聚焦多维评分，这里先手动给出 semantic_score
# 在完整项目中，semantic_score 可以由“小案例二”的 TF-IDF 分
# 和“小案例三”的 Word2Vec 分加权合成
# resumes 是简历列表，每个字典表示一名学生
resumes = [
    {
        "resume_name": "张同学",
        "skills": ["python", "sql", "数据分析", "可视化"],
        "education": "本科",
        "experience_years": 0,
    },
    {
        "resume_name": "李同学",
        "skills": ["java", "spring", "mysql", "接口开发"],
```



```

        "education": "本科",
        "experience_years": 1,
    },
]

# jobs 是岗位列表, 每个字典表示一个岗位
# semantic_scores 是为了教学方便预先写好的语义分:
# 例如 "张同学": 82 表示张同学与该岗位的语义相似度分为 82。
jobs = [
    {
        "job_name": "数据分析实习生",
        "required_skills": ["python", "sql", "数据分析", "excel"],
        "min_education": "本科",
        "min_experience_years": 0,
        "semantic_scores": {
            "张同学": 82,
            "李同学": 10,
        },
    },
    {
        "job_name": "后端开发实习生",
        "required_skills": ["java", "spring", "mysql"],
        "min_education": "大专",
        "min_experience_years": 1,
        "semantic_scores": {
            "张同学": 15,
            "李同学": 88,
        },
    },
]

```

rows 用来收集所有“学生-岗位”的匹配结果

```
rows = []

# 外层循环遍历每份简历
for resume in resumes:
    # 内层循环遍历每个岗位
    # 两层循环合起来，就能得到“每份简历 × 每个岗位”的所有组合
    for job in jobs:
        # 计算技能分，同时得到共同技能和缺少技能
        skill_score, matched_skills, missing_skills = calculate_skill_score(
            resume["skills"],
            job["required_skills"]
        )

        # 计算学历分
        education_score = calculate_education_score(
            resume["education"],
            job["min_education"]
        )

        # 计算经验分
        experience_score = calculate_experience_score(
            resume["experience_years"],
            job["min_experience_years"]
        )

        # 取出当前学生与当前岗位的语义分
        semantic_score = job["semantic_scores"][resume["resume_name"]]

        # 把各维度分数合成为综合分
        total_score = calculate_total_score(
            skill_score,
            semantic_score,
```

```

        education_score,
        experience_score
    )

    # 把当前这一个“学生-岗位”的结果保存成一行
    rows.append({
        "学生": resume["resume_name"],
        "岗位": job["job_name"],
        "技能分": round(skill_score, 2),
        "语义分": round(semantic_score, 2),
        "学历分": round(education_score, 2),
        "经验分": round(experience_score, 2),
        "综合分": round(total_score, 2),
        "共同技能": "、".join(matched_skills),
        "缺少技能": "、".join(missing_skills),
    })

# 把列表转换成 DataFrame, 方便排序、保存和页面展示
result_df = pd.DataFrame(rows)

# 按学生和综合分排序
result_df = result_df.sort_values(by=["学生", "综合分"], ascending=[True, False])

print(result_df)

```

运行结果解读

你会看到一个评分表，其中每一行代表：

某个学生 与 某个岗位 的匹配结果

表格中不仅有综合分，还有各个维度的分数和技能解释。

这张表后面可以直接用于：

- 保存为 CSV；
- 展示到 Streamlit；
- 给学生做答辩截图；
- 分析为什么某个岗位排在前面。

十八、案例 8：调整权重，看排序如何变化

权重不是随便写的。不同权重会影响推荐结果。

例如：

方案 A：技能优先

技能 50%，语义 25%，学历 15%，经验 10%

方案 B：文本相似度优先

技能 30%，语义 50%，学历 10%，经验 10%

同一份数据在不同方案下，排序可能不同。

代码清单： `08_weight_adjust_compare.py`

```
# 08_weight_adjust_compare.py
```

```
# 目标：比较不同权重方案对综合分的影响
```

```
def calculate_total_score(scores, weights):
```

```
    """
```

```
    根据分数字典和权重字典计算综合分。
```

```
    scores 示例:
```

```
        {"skill": 75, "semantic": 80, "education": 100, "experience": 70}
```

```
    weights 示例:
```

```
        {"skill": 0.4, "semantic": 0.3, "education": 0.15, "experience": 0.15}
```

```
    """
```

```
    total = 0
```

```
    # 遍历每一个评分维度, 例如 skill、semantic、education、experience
```

```
    for key in scores:
```

```
        # 当前维度得分 × 当前维度权重, 然后累加到总分
```

```
        total += scores[key] * weights[key]
```

```
    return total
```

```
# 同一组维度分数
```

```
scores = {
```

```
    "skill": 75,
```

```
    "semantic": 90,
```

```
    "education": 100,
```

```
    "experience": 40,
```

```
}
```

```
# 方案 A 更重视技能
weight_plan_a = {
    "skill": 0.50,
    "semantic": 0.25,
    "education": 0.15,
    "experience": 0.10,
}

# 方案 B 更重视语义相似度
weight_plan_b = {
    "skill": 0.30,
    "semantic": 0.50,
    "education": 0.10,
    "experience": 0.10,
}

# 用两套权重分别计算总分，观察结果差异
score_a = calculate_total_score(scores, weight_plan_a)
score_b = calculate_total_score(scores, weight_plan_b)

print("方案 A, 技能优先, 总分: ", round(score_a, 2))
print("方案 B, 语义优先, 总分: ", round(score_b, 2))
```

讨论题 4

权重应该由谁决定？

参考答案：

教学项目中可以由教师给出默认权重，学生再尝试调整。真实业务中，权重通常需要结合业务经验、历史数据、专家意见和实际效果一起确定。

十九、案例 9：综合案例，带解释的简历岗位匹配器

这个案例把前面内容合在一起：

1. 使用 TF-IDF + 余弦相似度计算词面相似度分；
2. 使用 Word2Vec 训练词向量，并计算句向量相似度分；
3. 把 TF-IDF 分和 Word2Vec 分合成为语义分；
4. 计算技能分；
5. 计算学历分；
6. 计算经验分；
7. 计算综合分；
8. 生成推荐理由；
9. 输出每个学生最匹配的岗位排序。

代码清单： `09_integrated_explainable_matcher.py`

```
# 09_integrated_explainable_matcher.py
# 目标：实现一个可解释的简历岗位匹配小系统

import pandas as pd
import numpy as np
from gensim.models import Word2Vec
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 学历等级映射表，用数字表示学历高低
education_level = {
    "高中": 1,
    "大专": 2,
    "本科": 3,
    "硕士": 4,
    "博士": 5,
}

def calculate_skill_score(resume_skills, job_required_skills):
    """
    计算技能匹配分，并返回共同技能和缺少技能。
    """
    # 转成集合后，可以用 & 求交集，用 - 求差集
    resume_set = set(resume_skills)
    job_set = set(job_required_skills)

    # 如果岗位没有填写技能要求，避免除以 0
    if len(job_set) == 0:
        return 0, set(), set()
```



```
# matched 是共同技能, missing 是岗位要求但学生没有的技能
matched = resume_set & job_set
missing = job_set - resume_set
```

```
# 技能分 = 共同技能数量 / 岗位要求技能数量
score = len(matched) / len(job_set) * 100
```

```
return score, matched, missing
```

```
def calculate_education_score(resume_education, job_min_education):
```

```
    """
```

```
    计算学历匹配分。
```

```
    """
```

```
    # 把文字学历转换成数字等级, 未知学历默认按 0 处理
```

```
    resume_level = education_level.get(resume_education, 0)
```

```
    job_level = education_level.get(job_min_education, 0)
```

```
    # 学生学历达到岗位最低要求, 给满分
```

```
    if resume_level >= job_level:
```

```
        return 100
```

```
    # 只差一个学历等级, 给部分分
```

```
    if job_level - resume_level == 1:
```

```
        return 60
```

```
    # 差距更大, 给较低分
```

```
    return 30
```

```
def calculate_experience_score(resume_years, job_min_years):
```

```
    """
```

计算经验匹配分。

"""

达到岗位要求, 给满分

```
if resume_years >= job_min_years:  
    return 100
```

gap 表示还差多少年经验

```
gap = job_min_years - resume_years
```

根据经验差距给不同分数

```
if gap == 1:  
    return 70  
elif gap == 2:  
    return 40  
else:  
    return 20
```

```
def calculate_city_score(resume_city, job_city):
```

"""

城市完全相同得 100 分, 否则给 60 分。

这里简单处理, 真实项目可加入是否接受异地。

"""

城市一致, 给满分

```
if resume_city == job_city:  
    return 100
```

城市不一致但不直接判 0, 因为学生可能愿意调整城市

```
return 60
```

```
def calculate_sentence_vector(text, model):
```

```
"""
```

把一段文本转换成句向量。

思路：

1. text 已经是用空格分隔好的分词文本。
2. 先取出每个词的 Word2Vec 向量。
3. 再用所有词向量的平均值表示整段文本。

注意：

这种“词向量求平均”的方法很简单，适合教学入门。
真实项目中还可以使用更复杂的句向量模型。

```
"""
```

```
# split() 会按照空格把文本切成词列表
```

```
words = text.split()
```

```
# 保存当前文本中每个有效词的向量
```

```
vectors = []
```

```
for word in words:
```

```
    # model.wv 表示 Word2Vec 训练后的词向量表
```

```
    # 如果词在词向量表中，才取出它的向量
```

```
    if word in model.wv:
```

```
        vectors.append(model.wv[word])
```

```
# 如果这段文本没有任何词能找到向量，就返回一个全 0 向量
```

```
if len(vectors) == 0:
```

```
    return np.zeros(model.vector_size)
```

```
# np.mean(vectors, axis=0) 表示按列求平均
```

```
# 结果仍然是一个 vector_size 维的向量
```

```
return np.mean(vectors, axis=0)
```

```
def calculate_word2vec_similarity_matrix(resume_texts, job_texts):  
    """  
    训练 Word2Vec 模型，并计算所有简历与所有岗位的 Word2Vec 相似度矩阵。  
    """  
    # 把所有文本放在一起作为训练语料  
    documents = resume_texts + job_texts  
  
    # Word2Vec 需要的训练格式是：二维列表  
    # 每一行是一段文本的词列表，例如 ["python", "sql", "数据分析"]  
    tokenized_documents = [text.split() for text in documents]  
  
    # 训练一个很小的 Word2Vec 模型  
    # vector_size: 每个词向量的维度  
    # window: 观察上下文时向前向后看的词数  
    # min_count=1: 只出现 1 次的词也参与训练，适合教学小数据  
    # sg=1: 使用 Skip-gram 模型  
    # seed: 固定随机种子，让每次运行结果更稳定  
    model = Word2Vec(  
        sentences=tokenized_documents,  
        vector_size=50,  
        window=3,  
        min_count=1,  
        workers=1,  
        sg=1,  
        seed=42  
    )  
  
    # 把每份简历文本转换成句向量  
    resume_sentence_vectors = np.array([  
        calculate_sentence_vector(text, model)  
        for text in resume_texts
```

```

])

# 把每个岗位文本转换成句向量
job_sentence_vectors = np.array([
    calculate_sentence_vector(text, model)
    for text in job_texts
])

# 使用余弦相似度比较“简历句向量”和“岗位句向量”
return cosine_similarity(resume_sentence_vectors, job_sentence_vectors)

def calculate_total_score(
    skill_score,
    semantic_score,
    education_score,
    experience_score,
    city_score
):
    """
    综合评分。

    本案例加入城市分，所以权重设计为：
    - 技能分：35%
    - 语义分：30%
    - 学历分：15%
    - 经验分：15%
    - 城市分：5%
    """
    total = (
        # 技能是核心维度，占 35%
        skill_score * 0.35

```

```

    # 语义分来自 TF-IDF + Word2Vec 的合成结果
    + semantic_score * 0.30
    # 学历、经验都是结构化门槛信息
    + education_score * 0.15
    + experience_score * 0.15
    # 城市作为附加因素，占比较小
    + city_score * 0.05
)

return total

def generate_reason(
    matched_skills,
    missing_skills,
    skill_score,
    semantic_score,
    education_score,
    experience_score,
    city_score
):
    """
    根据各个维度的分数生成推荐理由。
    """
    # reasons 用列表保存多条解释，最后再拼接成一句完整说明
    reasons = []

    # 解释技能匹配情况
    if skill_score >= 80:
        reasons.append(f"技能匹配较高，共同技能: {'，'.join(matched_skills)}")
    elif skill_score >= 50:
        reasons.append(f"技能部分匹配，共同技能: {'，'.join(matched_skills)}")

```

```
else:
    reasons.append("技能匹配偏低, 需要补充岗位核心技能")

if len(missing_skills) > 0:
    reasons.append(f"缺少技能: {'', '.join(missing_skills)}")

# 解释语义相似度情况
if semantic_score >= 80:
    reasons.append("文本相似度较高, 简历内容与岗位方向接近")
elif semantic_score >= 50:
    reasons.append("文本相似度中等, 简历与岗位存在一定相关性")
else:
    reasons.append("文本相似度较低, 简历内容与岗位描述差异较大")

# 解释学历是否满足岗位要求
if education_score == 100:
    reasons.append("学历满足岗位要求")
else:
    reasons.append("学历未完全满足岗位要求")

# 解释经验是否满足岗位要求
if experience_score == 100:
    reasons.append("经验年限满足岗位要求")
elif experience_score >= 70:
    reasons.append("经验略低于岗位要求, 但差距不大")
else:
    reasons.append("经验年限与岗位要求差距较大")

# 解释城市是否匹配
if city_score == 100:
    reasons.append("城市匹配")
else:
```

```
reasons.append("城市不完全匹配")
```

```
return "; ".join(reasons)
```

```
# 简历列表, 每个字典表示一名学生
```

```
# text 字段是已经整理过的分词文本, 方便直接交给 TfidfVectorizer
```

```
resumes = [  
    {  
        "resume_name": "张同学",  
        "skills": ["python", "sql", "数据分析", "可视化"],  
        "education": "本科",  
        "experience_years": 0,  
        "city": "南昌",  
        "text": "python sql 数据分析 可视化 课程项目 报表"  
    },  
    {  
        "resume_name": "李同学",  
        "skills": ["java", "spring", "mysql", "接口开发"],  
        "education": "本科",  
        "experience_years": 1,  
        "city": "南昌",  
        "text": "java spring mysql 后端开发 接口 项目"  
    },  
    {  
        "resume_name": "王同学",  
        "skills": ["linux", "shell", "服务器", "部署"],  
        "education": "大专",  
        "experience_years": 1,  
        "city": "九江",  
        "text": "linux shell 服务器 部署 网络 运维"  
    },  
]
```



```
]

# 岗位列表, 每个字典表示一个岗位
jobs = [
    {
        "job_name": "数据分析实习生",
        "required_skills": ["python", "sql", "数据分析", "excel"],
        "min_education": "本科",
        "min_experience_years": 0,
        "city": "南昌",
        "text": "python sql 数据分析 报表 可视化 excel"
    },
    {
        "job_name": "后端开发实习生",
        "required_skills": ["java", "spring", "mysql"],
        "min_education": "大专",
        "min_experience_years": 1,
        "city": "南昌",
        "text": "java spring mysql 接口开发 后端服务"
    },
    {
        "job_name": "运维实习生",
        "required_skills": ["linux", "shell", "服务器"],
        "min_education": "大专",
        "min_experience_years": 0,
        "city": "九江",
        "text": "linux shell 服务器 网络 部署 运维"
    },
]
```

```
# 第一步: 准备语义相似度计算需要的文本
# 先分别取出所有简历文本和所有岗位文本
```

```
resume_texts = [resume["text"] for resume in resumes]
job_texts = [job["text"] for job in jobs]

# 简历文本放前面，岗位文本放后面，便于后面切片
documents = resume_texts + job_texts

# 创建 TF-IDF 向量化工具
vectorizer = TfidfVectorizer()

# fit_transform 会学习词表，并把每段文本转换成 TF-IDF 向量
tfidf_matrix = vectorizer.fit_transform(documents)

# 前 len(resumes) 行是简历向量
resume_vectors = tfidf_matrix[:len(resumes)]

# 后 len(resumes) 行开始是岗位向量
job_vectors = tfidf_matrix[len(resumes):]

# tfidf_similarity_matrix 的行表示简历，列表示岗位
# 值越大，说明词面相似度越高
tfidf_similarity_matrix = cosine_similarity(resume_vectors, job_vectors)

# 第三步：计算 Word2Vec 相似度
# 这里会使用当前示例语料训练一个小型 Word2Vec 模型
word2vec_similarity_matrix = calculate_word2vec_similarity_matrix(
    resume_texts,
    job_texts
)

# 第四步：计算多维评分
rows = []
```

```

# 外层循环遍历每一份简历
for resume_index, resume in enumerate(resumes):
    # 内层循环遍历每一个岗位
    for job_index, job in enumerate(jobs):
        # TF-IDF 相似度在 0 到 1 之间, 乘以 100 转成百分制
        tfidf_score = tfidf_similarity_matrix[resume_index][job_index] * 100

        # Word2Vec 余弦相似度理论上可能为负数
        # 教学评分中我们把负相似度按 0 处理, 再转成百分制
        word2vec_score = max(
            word2vec_similarity_matrix[resume_index][job_index],
            0
        ) * 100

        # 合成语义分:
        # TF-IDF 占 60%, Word2Vec 占 40%
        semantic_score = tfidf_score * 0.60 + word2vec_score * 0.40

        # 计算技能分, 并得到共同技能、缺少技能
        skill_score, matched_skills, missing_skills = calculate_skill_score(
            resume["skills"],
            job["required_skills"]
        )

        # 计算学历分
        education_score = calculate_education_score(
            resume["education"],
            job["min_education"]
        )

        # 计算经验分
        experience_score = calculate_experience_score(

```

```
        resume["experience_years"],
        job["min_experience_years"]
    )

    # 计算城市分
    city_score = calculate_city_score(
        resume["city"],
        job["city"]
    )

    # 把所有维度合成为综合分
    total_score = calculate_total_score(
        skill_score,
        semantic_score,
        education_score,
        experience_score,
        city_score
    )

    # 根据各维度结果生成推荐理由
    reason = generate_reason(
        matched_skills,
        missing_skills,
        skill_score,
        semantic_score,
        education_score,
        experience_score,
        city_score
    )

    # 保存当前“学生-岗位”的一行结果
    rows.append({
```

```
        "学生": resume["resume_name"],
        "岗位": job["job_name"],
        "综合分": round(total_score, 2),
        "技能分": round(skill_score, 2),
        "TF-IDF分": round(tfidf_score, 2),
        "Word2Vec分": round(word2vec_score, 2),
        "语义分": round(semantic_score, 2),
        "学历分": round(education_score, 2),
        "经验分": round(experience_score, 2),
        "城市分": round(city_score, 2),
        "推荐理由": reason,
    })

# 转换成 DataFrame, 方便排序、保存 CSV 或展示到 Streamlit
result_df = pd.DataFrame(rows)

# 每个学生内部按综合分从高到低排序
result_df = result_df.sort_values(by=["学生", "综合分"], ascending=[True, False])

print(result_df)

# 如果要保存给 Streamlit 展示, 可以取消下面这行注释
# result_df.to_csv("explainable_match_result.csv", index=False, encoding="utf-8-sig")
```

运行结果应该怎么看

输出表格中, 每一行都有:

- 学生;
- 岗位;
- 综合分;

- 技能分;
- TF-IDF分;
- Word2Vec分;
- 语义分;
- 学历分;
- 经验分;
- 城市分;
- 推荐理由。

这比单纯输出相似度更适合做项目展示。

其中:

- TF-IDF分 更关注“词面是否重合”，例如 `python`、`sql` 是否同时出现;
- Word2Vec分 更关注“上下文方向是否接近”，例如 `数据分析`、`报表`、`可视化` 是否经常出现在相近语境中;
- 语义分 是把前两者加权合成后的结果，后面综合评分只使用这个统一的语义分。

需要注意：本案例中的 Word2Vec 只用几条示例文本训练，效果主要用于教学演示。真实项目中应该使用更多简历和岗位文本训练，或者加载更大规模的预训练词向量。

比如你可以在答辩中说明：

张同学推荐数据分析实习生，不只是因为文本相似度高，还因为技能、学历、城市都比较匹配。

课堂讨论题 1

为什么综合案例中没有直接让语义相似度决定最终排序？

参考答案：

因为语义相似度只能反映文本方向是否接近，不能完整判断学历、经验、城市等实际条件。综合评分可以把文本信息和结构化规则结合起来，结果更接近真实招聘场景。

二十、多维评分的常见问题

1. 权重是不是越复杂越好

不是。

初学阶段建议先使用简单、清楚、能解释的权重。

如果权重太复杂，学生可能跑出了结果，却讲不清楚为什么这样算。

2. 规则分是不是比 AI 算法低级

不是。

在很多真实系统中，规则和模型会一起使用。

例如：

- 用 TF-IDF 或模型计算文本相似度；
- 用规则判断学历、经验、城市；
- 用权重或排序模型合成最终结果；
- 用解释模板说明推荐原因。

规则的优点是透明、稳定、容易调试。

3. 如果某些字段缺失怎么办

常见处理方式有：

- 给默认分；
- 跳过该维度；
- 提示数据不完整；
- 在结果解释中说明“该字段缺失，未参与评分”。

教学项目中，建议先保证数据字段完整，再讨论缺失值处理。

4. 推荐理由会不会过于模板化

会有这种可能。

但教学阶段使用模板解释是很合适的，因为它清楚、稳定、便于学生理解。

后续如果想升级，可以尝试：

- 更细致的规则解释；
- 根据缺少技能给学习建议；
- 用大语言模型生成自然语言解释；
- 给出岗位提升路径。

二十一、课堂练习

练习 1：修改技能评分

请修改 01_skill_score_demo.py :

```
# 学生简历中写到的技能
resume_skills = ["python", "sql", "tableau", "统计分析"]

# 岗位要求的核心技能
job_required_skills = ["python", "sql", "数据分析", "excel"]
```

问题:

1. 共同技能有哪些?
2. 缺少技能有哪些?
3. 技能匹配分是多少?

参考答案:

共同技能是 python、sql。缺少技能是 数据分析、excel。岗位要求 4 个技能, 匹配 2 个, 所以技能匹配分是 50 分。

练习 2: 解释学历评分

岗位要求硕士, 学生学历是本科。

按照本讲义的规则, 学历分是多少? 为什么?

参考答案:

学历分是 60。因为本科比硕士低一级, 属于“低 1 级”的情况。

练习 3: 解释经验评分

岗位要求 3 年经验, 学生有 1 年经验。

按照本讲义规则，经验分是多少？为什么不直接给 0？

参考答案：

经验分是 40。因为学生比岗位要求少 2 年。没有直接给 0，是因为经验不足不代表完全无法胜任，尤其还可以结合技能、学历、项目经历等维度综合判断。

练习 4：计算综合分

已知：

技能分：80
语义分：70
学历分：100
经验分：70

使用权重：

技能 40%，语义 30%，学历 15%，经验 15%

综合分是多少？

参考答案：

$80 \times 0.40 = 32$
 $70 \times 0.30 = 21$
 $100 \times 0.15 = 15$
 $70 \times 0.15 = 10.5$
综合分 = 78.5

练习 5：写一条推荐理由

某学生与岗位的结果如下：

共同技能：python、sql
缺少技能：excel
语义分：76
学历分：100
经验分：70
城市分：100

请写出一条推荐理由。

参考答案：

该学生与岗位存在较好匹配，共同技能包括 Python 和 SQL，学历满足岗位要求，城市匹配；但仍缺少 Excel 技能，经验略低于岗位要求，建议补充相关工具练习后再投递。

二十二、项目扩展任务

如果你已经完成了本节课所有案例，可以继续尝试下面的扩展。

扩展 1：加入证书权重

把综合分改成：

技能 35%，语义 25%，学历 15%，经验 15%，证书 10%

要求：

- 计算证书分；
- 加入综合分；
- 在推荐理由中说明匹配证书。

参考思路：

参考 `04_extra_rule_score_demo.py` 中的 `calculate_certificate_score()`，再修改 `calculate_total_score()` 的参数和权重。

扩展 2：加入“最低门槛”

如果技能分低于 30，则不推荐该岗位。

参考思路：

在排序前增加判断，如果 `skill_score < 30`，可以把综合分降低，或者在结果中标记为“不推荐”。

扩展 3：输出 Top-N 推荐结果

每个学生只输出综合分最高的前 2 个岗位。

参考思路：

可以在 `pandas` 中使用 `groupby("学生").head(2)`，前提是已经按学生和综合分排好序。

示例：

```
# 第一步：先按学生和综合分排序
sorted_df = result_df.sort_values(
    by=["学生", "综合分"],
    ascending=[True, False]
)

# 第二步：每个学生只保留前 2 行，也就是 Top 2 推荐岗位
top2_df = sorted_df.groupby("学生").head(2)
```

二十三、本节课小结

本节课我们完成了从“文本相似度”到“可解释匹配”的升级。

你需要掌握：

知识点	你应该能说清楚什么
技能匹配分	如何计算共同技能和缺少技能
学历分	如何把学历转换成可比较等级
经验分	如何根据经验差距给分
附加规则	城市、薪资、证书如何参与评分
综合权重	多个维度如何合成总分
可解释匹配	如何生成推荐理由
批量匹配	如何用 pandas 输出评分表

请记住本节课最重要的一句话：

好的匹配系统不只告诉你“谁排第一”，还要告诉你“为什么排第一”。

下一步，我们会把这些结果展示到 Streamlit 页面中，让用户选择某个学生简历后，实时看到右侧推荐岗位、综合分和推荐理由。